



ЛЕКЦИЯ 15

Анализ текстов и журналов событий

от TF-IDF к эмбедингам

Машинное обучение и безопасность систем

Магистратура НИУ ВШЭ (МИЭМ) · ОП «Информационная безопасность и технологии ИИ»

Автор: Силаев Ю. В. Время изучения: ~65 минут · 53 слайда



Эта лекция за одну минуту

2 / 53

О чём пойдёт речь и зачем это инженеру по безопасности

Значительная часть данных информационной безопасности имеет текстовую или полутекстовую природу: сообщения и письма, URL и доменные имена, командные строки процессов, события журналов, описания алертов, тикеты и отчёты SOC. Чтобы применить к ним машинное обучение, текст нужно превратить в числа – и именно выбор этого представления определяет, какие закономерности модель вообще способна увидеть.

1

Разреженные представления

Bag-of-words, n-grams и TF-IDF.

Линейная модель на TF-IDF – сильный и интерпретируемый baseline.

2

Логи как особый текст

У лог-сообщений есть скрытая структура: шаблон плюс переменные параметры. Их разбор снижает шум.

3

Плотные представления

Embeddings и BERT-подобные модели на уровне применения – когда они оправданы, а когда избыточны.

Ключевой тезис: выбор представления текста определяет качество, интерпретируемость и поведение модели при смене словаря, домена или источника логов.



Учебные результаты лекции

- выделять основные этапы подготовки текстовых данных;
- различать bag-of-words, n-grams, TF-IDF и embeddings;
- объяснять, почему линейная модель на TF-IDF – сильный baseline для текстов;
- выбирать базовое представление текста под задачу классификации или поиска;
- объяснять особенности текстов и логов как данных безопасности;
- распознавать риски: drift словаря, шум разметки, доменная специфика;
- интерпретировать важные токены и ошибочные примеры (инструментарий из L09);
- понимать роль transformer-based embeddings на уровне применения – без архитектуры.



Что нужно знать до начала

4 / 53

Эта лекция опирается на пять предыдущих

L15 не вводит машинное обучение с нуля – она применяет уже известные инструменты к текстам и логам. Перед изучением убедитесь, что владеете материалом этих лекций:

L06

Линейные модели Логистическая регрессия, log-loss, веса w . Именно линейка станет нашим baseline на TF-IDF.

L09

Интерпретируемость Анализ важных признаков и ошибок. Перенесём на анализ важных токенов текстовой модели.

L10

Pipeline Воспроизводимые конвейеры и защита от утечки на этапе препроцессинга.

L12

Логи и время Логи как события с временной структурой. Здесь возьмём их текстовую сторону.

L13

Основы нейросетей Базовое устройство нейросети. Понадобится при разговоре о BERT на уровне применения.



План лекции

5 / 53

7 частей · 53 слайда · ~65 минут изучения

Часть	Название	Слайды	Время
1	Введение	1-5	~6 мин
2	Мотивация: текст и логи в задачах ИБ	6-9	~5 мин
3	Основное содержание (4 блока)	10-43	~40 мин
4	Связки с практикой ИБ	44-48	~7 мин
5	Итоги	49-51	~4 мин
6	Источники для углубления	52	~2 мин
7	Глоссарий	53	~1 мин

Часть 3 – ядро лекции: четыре смысловых блока от токенизации до embeddings.



Большая часть сигнала в безопасности приходит в виде строк

Часть 2 · Мотивация

Когда мы думаем о машинном обучении в безопасности, первым на ум приходит сетевой трафик в виде таблиц чисел. Но огромная доля повседневной работы аналитика – это чтение строк: писем, ссылок, команд, сообщений журналов и описаний инцидентов. Всё это – текст или полутекст, и без числового представления ни одна модель его не обработает.

Фишинговые письма

тема, тело, заголовки

URL и домены

пути, поддомены, TLD

Командные строки

процессы, аргументы, флаги

События журналов

шаблон + параметры

Описания алертов

правила и сигнатуры

Тикеты и отчёты SOC

свободный текст аналитика

Имена файлов и хостов

артефакты атаки

Сообщения и чаты

социальная инженерия

CIC-IDS-2018 – табличный датасет курса – здесь не главный. Текстовые задачи живут на своих данных: письма, URL, команды.



ИБ-кейс: фишинговое письмо глазами модели

7 / 53

Какие токены несут сигнал, а какие – шум

Часть 2 · Мотивация

ИБ-КЕЙС

Фрагмент письма (упрощённо):

Subject: Ваш аккаунт будет заблокирован!
Срочно подтвердите данные в течение 24 часов,
иначе доступ будет приостановлен.
Перейдите: <http://secure-payra1.com/login>

Сигнальные токены: «срочно», «заблокирован», «24 часа», «подтвердите данные», подозрительный домен раура1 (цифра 1 вместо буквы l).

Что увидит модель

● Слова (bag-of-words)

ловит «срочно», «заблокирован» – лексику давления

● Символьные n-grams

ловит раура1: подмену символа в домене

● Структуру URL

редирект, странный TLD, отсутствие https

Один и тот же текст несёт сигнал на нескольких уровнях сразу – выбор представления решает, какой из них модель заметит.

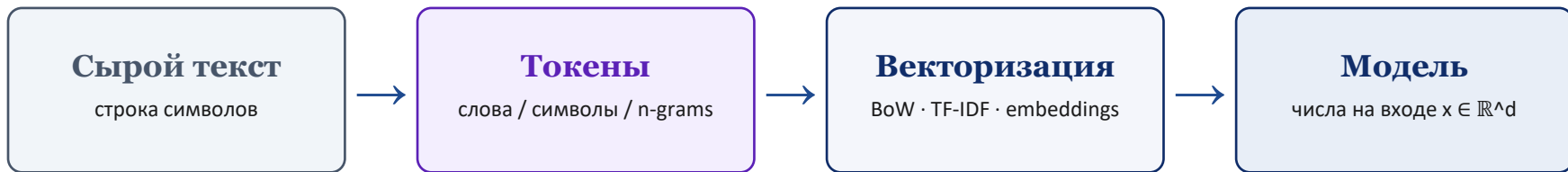


Текст не подавать в модель напрямую

Центральный вопрос лекции

Часть 2 · Мотивация

Модели машинного обучения работают с числами: вектор признаков $x \in \mathbb{R}^d$, на который умножаются веса. Строка «срочно подтвердите данные» – это не вектор. Поэтому между сырым текстом и моделью всегда стоит шаг векторизации: превращение текста в числовое представление.



Где проходит граница лекции

Шаг «векторизация» – это и есть тема L15. Мы пройдем его от простого (счётчики слов) к сложному (плотные эмбединги) и научимся выбирать представление под задачу, объём данных и требования к интерпретируемости.



Что будет, если выбрать представление наугад

Цена ошибки на шаге векторизации

Часть 2 · Мотивация

⚠ ЧАСТАЯ ОШИБКА

«Возьму векторизацию по умолчанию – какая разница, всё равно модель разберётся»

Разница огромная. Неудачное представление либо стирает сигнал атаки, либо приклеивает модель к артефактам конкретного набора данных. Модель «разберётся» – но выучит не то, что нужно, и тихо развалится в проде.

ПОЧЕМУ ЭТО НЕВЕРНО

Чрезмерная очистка

Удалили цифры и спецсимволы – и `раура1` стало неотличимо от `раурал`. Сигнал атаки потерян.

Drift словаря

Новая кампания приносит новые слова. Модель их не видела на `train` – метрики падают молча.

Утечка через ID

В тексте остался идентификатор кампании или результата расследования – модель «угадывает» по нему.



БЛОК 1

От строки к токенам

Прежде чем считать слова и строить векторы, нужно решить более раннюю задачу: на какие единицы разбить текст. Этот выбор называется токенизацией – и он не так очевиден, как кажется.

- Текст как последовательность токенов
- Уровни токенизации: слова, символы, n-grams
- Char n-grams для URL и команд
- Очистка и риск потерять сигнал



Текст как последовательность ТОКЕНОВ

Базовая единица любого текстового представления

Часть 3 · Блок 1 · Токенизация

Для модели документ – это не сплошная строка, а последовательность отдельных единиц, токенов. Токен – минимальный кусок текста, который мы считаем отдельным элементом: чаще всего слово, но это может быть и символ, и часть слова, и целая n-грамма. Всё дальнейшее – счётчики, веса, эмбединги – строится поверх выбранных токенов. Поэтому токенизация – первый и часто недооценённый шаг.

Пример: одна строка – три способа разбить

```
login failed for user admin from 10.0.0.5
```

По словам [login] [failed] [for] [user] [admin] [from] [10.0.0.5]

По символам [l][o][g][i][n][][f][a][i][l][e][d] ...

Биграммы слов [login failed] [failed for] [for user] [user admin] ...



Слова, символы и *n-grams*

ОПРЕДЕЛЕНИЕ

n-грамма — это последовательность из n подряд идущих токенов. **Word n-gram** составляется из n подряд идущих слов, **character n-gram** — из n подряд идущих символов. При $n = 1$ это просто отдельные слова или символы (unigram).

Слова (word)

«login», «failed»

Естественно для текста на языке. Теряет внутреннюю структуру слова.

Символы (char)

«l», «o», «g»...

Устойчив к опечаткам и новизне. Ловит структуру внутри токена.

n-grams

«login failed»

Возвращает часть порядка и контекста, потерянного мешком слов.



Character n-grams для URL и командных строк

Когда символы важнее слов

Часть 3 · Блок 1 · Токенизация

В обычном тексте слово – удобная единица. Но URL, домены и командные строки часто состоят из «слов», которых нет ни в одном словаре: случайные поддомены, обфусцированные пути, склеенные параметры. Словарный подход здесь беспомощен – почти каждый токен новый. Символьные n-граммы решают это: они ловят характерные сочетания символов, которые повторяются даже в невиданных ранее строках.

Домен:

secure-paypal.com

Char 3-grams:

sec · ecu · cur · ure · re- · e-p · -pa · pay · ayp
· ура · pa1 · a1. · 1.c · .co · com

Почему это ловит атаку

- Триграмма «pa1» – характерный след подмены l → 1 в paypal.
- Работает на доменах, которых модель никогда не видела целиком.
- Те же n-граммы ловят флаги и пути в командных строках процессов.



Очистка текста и риск переочистки

14 / 53

Стандартный препроцессинг может стереть сигнал атаки

Часть 3 · Блок 1 · Токенизация

В классическом NLP текст принято «чистить»: приводить к нижнему регистру, удалять цифры, пунктуацию и спецсимволы, отбрасывать стоп-слова. Для новостей и отзывов это разумно. Для данных безопасности – опасно.

⚠ ЧАСТАЯ ОШИБКА

«Применю стандартную очистку – так делают во всех NLP-туториалах»

Те самые символы, которые туториал удаляет как «мусор», в ИБ часто и есть признак атаки: цифры в домене, точки в IP, слэши в путях, регистр в командах.

ПОЧЕМУ ЭТО НЕВЕРНО

До очистки: secure-paypa1.com/login → **После «очистки»:** securepaypacomlogin

Удалили дефис, цифру 1 и точки – и подмена l→1, и структура URL исчезли. Модель больше не отличит фишинг от легитимного домена.



1

Токен $\neq$ слово

Токен – любая выбранная единица текста: слово, символ или n-грамма. Выбор делается до всего остального.

2

Три уровня токенизации

Слова дают естественность, символы – устойчивость к новизне, n-grams возвращают часть порядка.

3

Char n-grams для технических строк

URL, домены и команды состоят из несловарных токенов – символьные n-граммы ловят сигнал там, где словарь бессилён.

4

Очистка $\neq$ всегда польза

Стандартный NLP-препроцессинг может стереть именно те символы, которые в ИБ являются признаком атаки.

Дальше – блок 2: как из токенов получить числовой вектор и почему линейная модель на TF-IDF – сильный baseline.



БЛОК 2

Считаем слова правильно

Токены выбраны – пора превратить их в числа. Начнём с самого простого счёта слов, увидим его слабые места, и придём к TF-IDF: взвешиванию, которое отделяет информативные токены от фоновых.

- Bag-of-words и его ограничения
- Интуиция и формула TF-IDF
- Word n-grams и character n-grams в TF-IDF
- Дальше (часть 2 блока) – линейный baseline и важные токены



Самый простой способ превратить текст в вектор

ОПРЕДЕЛЕНИЕ

Bag-of-words (мешок слов) – представление документа вектором, где каждая координата соответствует одному слову словаря, а её значение – сколько раз это слово встретилось в документе. Порядок слов отбрасывается: документ – это «мешок», а не последовательность.

Пример: словарь из 5 слов, два сообщения

Документ	срочно	аккаунт	счёт	оплата	спасибо
«Срочно! Подтвердите аккаунт, аккаунт заблокирован»	1	2	0	0	0
«Спасибо за оплату счёта»	0	0	1	1	1

Каждое сообщение стало вектором из 5 чисел. Слово «аккаунт» дважды → координата 2.



Почему простого счёта недостаточно

Bag-of-words прост и удивительно силён как старт, но у него есть три системные слабости. Все они вытекают из того, что счёт не различает, насколько слово информативно – он лишь регистрирует, сколько раз оно встретилось.

1

Частые слова доминируют

«и», «в», «на» встречаются всюду и набирают большой счёт, заглушая редкие, но важные токены.

2

Нет веса редкости

Слово, встретившееся в одном документе из тысячи, считается наравне с фоновым. Его сигнальная ценность теряется.

3

Огромная размерность

Каждое уникальное слово – отдельная координата. Словарь на десятки тысяч слов даёт разреженные векторы той же длины.



Информативно то, что редко в корпусе, но часто в документе

Идея TF-IDF – заменить голый счёт на взвешенный. Токен ценен, если он часто встречается в данном документе (term frequency, TF), но при этом редок во всём корпусе (inverse document frequency, IDF). Слово, которое есть почти везде, получает низкий вес; слово, характерное для немногих документов, – высокий.

TF – term frequency

Как часто токен в этом документе

Чем чаще слово в документе, тем оно для него важнее.
Растёт со счётом внутри одного текста.

IDF – inverse document frequency

Насколько токен редок в корпусе

Чем в меньшем числе документов встречается слово, тем выше его различающая сила. Фоновые слова почти не дают вклада.

Вес = $TF \times IDF$. Высокий вес получают токены, одновременно частые в документе и редкие в корпусе – именно они лучше всего отличают один класс текстов от другого.



Та же интуиция, записанная точно

ФОРМУЛА

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log(N / \text{df}(t))$$

IDF – это логарифм отношения: всего документов N к числу документов $\text{df}(t)$, где встретился токен t

tf(t, d)

частота токена t в документе d – как часто слово в этом тексте

N

общее число документов в корпусе

df(t)

document frequency – в скольких документах встретился токен t

$\log(N / \text{df}(t))$

IDF: редкие токены → большой логарифм; почти всеобщие → близко к нулю



Один механизм взвешивания – разные единицы

Часть 3 · Блок 2 · TF-IDF

TF-IDF не привязан к словам: взвешивать можно любые токены, в том числе n-граммы. Выбор уровня меняет то, какой сигнал модель увидит. На практике в ИБ часто комбинируют оба уровня в одном векторе.

Word n-grams

- Биграммы/триграммы слов возвращают часть порядка: «подтвердите аккаунт» как единый признак.
- Хорошо работают на связном тексте: письма, тикеты, отчёты.
- Риск: словарь n-грамм растёт быстро, многие встречаются один раз.

Character n-grams

- Символьные n-граммы устойчивы к новизне и обфускации: ловят «pa1», «.exe», «/bin/».
- Незаменимы на URL, доменах, командных строках.
- Риск: теряют крупную семантику слов, дают много шумных признаков.

Практическое правило: для связного текста начинают со слов; для URL и команд добавляют символьные n-граммы.



Сильный и честный baseline для текстов

Возьмём векторы TF-IDF и подадим их в логистическую регрессию из L06. Этот простой конвейер – TF-IDF плюс линейная модель – на удивление часто оказывается очень сильным, а иногда и непобедимым решением для текстовой классификации. Он быстрый, дешёвый и прозрачный: веса модели прямо показывают, какие токены тянут предсказание в ту или иную сторону.

Линия курса · baseline-дисциплина. Логистическая регрессия – из L06. Как простое дерево было baseline для таблиц (L08), а MLP сравнивается с бустингом (L13), здесь TF-IDF + линейка – обязательная точка отсчёта перед эмбедингами.

Быстрый и дешёвый

Обучается за секунды на тысячах текстов, не требует GPU.

Прозрачный

Вес каждого токена виден напрямую – модель легко проверить.

Сильный по качеству

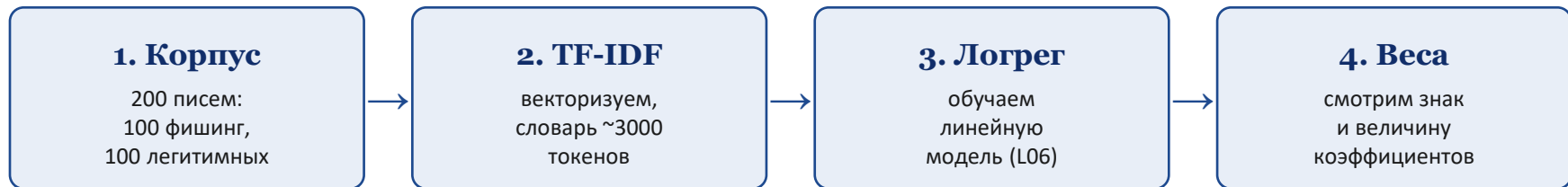
На многих ИБ-задачах достигает уровня, который сложные модели едва превосходят.



Пример: классификация фишинга на TF-IDF

От текста к весам – по шагам

Часть 3 · Блок 2 · TF-IDF



Результат: токены с наибольшим весом «в сторону фишинга»

Токен	Вес w	Класс, в сторону которого тянет
срочно	+1.84	фишинг
pa1 (char 3-gram)	+1.52	фишинг
заблокирован	+1.21	фишинг
спасибо	-0.97	легитимное
счёт-фактура	-1.30	легитимное



Веса линейной модели = карта сигнала

Прозрачность – главное прикладное преимущество TF-IDF + линейки. У линейной модели вес каждого токена напрямую читается: знак показывает класс, величина – силу влияния. Это та же интерпретация коэффициентов, что вводилась в L09 для признаков табличных моделей, только теперь признаки – токены текста.

Линия курса · интерпретируемость. Интерпретация важных токенов – продолжение L09. Коэффициенты линейной модели (L06) → анализ признаков (L09) → важные токены текста (здесь).

Проверка адекватности

Если в топе важных токенов – осмысленные признаки атаки (домены, лексика давления), модель учится правильному.

Раннее обнаружение утечки

Если в топе – идентификаторы кампаний, даты или артефакты разметки, модель «угадывает» по утечке, а не по сути.



Частая ошибка: embeddings без baseline

Сложное вместо простого – без проверки

Часть 3 · Блок 2 · TF-IDF

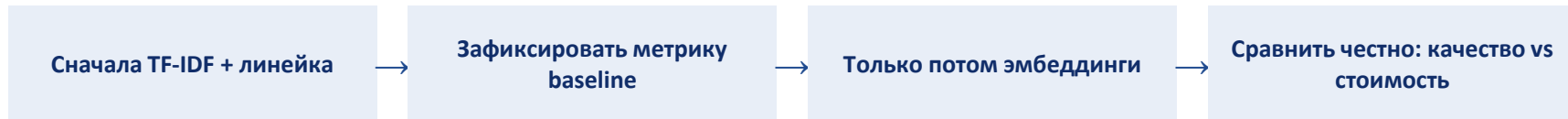
Самый частый методический промах в текстовых задачах – пропустить baseline и сразу взять тяжёлую модель.

⚠ ЧАСТАЯ ОШИБКА

«Сразу возьму эмбединги или BERT – это же современнее, чем какой-то TF-IDF»

Без baseline невозможно сказать, что эмбединги вообще что-то дали. Если простая линейка показывает $F1 = 0.94$, а тяжёлая модель – 0.945 , выигрыш не стоит роста стоимости, потери прозрачности и риска переобучения.

Правильный порядок





Что мы разобрали: TF-IDF и baseline

Мини-резюме блока 2

Часть 3 · Блок 2 · TF-IDF

1

Bag-of-words считает, TF-IDF взвешивает

TF-IDF поднимает токены, частые в документе и редкие в корпусе, и гасит фоновые слова.

2

TF-IDF + линейка – обязательный baseline

Простой, быстрый и часто очень сильный. Точка отсчёта перед любыми эмбедингами (линия baseline-дисциплины).

3

Веса токенов интерпретируемы

Знак и величина коэффициента читаются напрямую – это и проверка адекватности, и ранний детектор утечки (линия L09).

4

Эмбединги – только после baseline

Без зафиксированной метрики baseline нельзя оценить, дала ли сложная модель реальный выигрыш.

Дальше – блок 3: логи как особый текст. У лог-сообщений есть скрытая структура – шаблон плюс параметры.



БЛОК 3

Логи – особый текст

Лог-сообщения тоже текст – но у них есть скрытая структура, которой нет у писем. Каждая строка обычно состоит из постоянного шаблона и переменных параметров. Если это увидеть, текстовые методы работают намного точнее.

- Особенности логов: шаблоны, параметры, технические токены
- Log parsing – отделение шаблона от переменных
- Почему один шаблон может значить разное



Смешанная структура: текст плюс техника

Часть 3 · Блок 3 · Логи

Лог отличается от естественного текста. В нём перемешаны человекочитаемые слова и сугубо технические токены: коды ошибок, идентификаторы, пути, IP-адреса, команды. Часть строки повторяется из записи в запись (шаблон), часть меняется (параметры). Эта двойственность – ключ к работе с логами.

Технические токены, которые несут сигнал

Коды ошибок

0x80070005, E_FAIL

Идентификаторы

PID, session, UID

Пути

/etc/passwd, C:\Temp

IP-адреса

10.0.0.5, ::1

Команды

powershell -enc ...

Уровни/статусы

ERROR, DENIED, 403

Временная сторона логов – сессии, окна, drift – разбиралась в L12. Здесь нас интересует их текстовая структура.



Отделяем постоянное от переменного

Часть 3 · Блок 3 · Логи

ОПРЕДЕЛЕНИЕ

Log parsing – разбор лог-сообщения на постоянный **шаблон (template)** и переменные **параметры (variables)**.

Шаблон – это «скелет» сообщения, общий для многих записей; параметры – конкретные значения, подставленные в этот скелет.

Исходное сообщение

Failed login for user admin from 10.0.0.5

Шаблон: Failed login for user <*> from <*>

Параметры: admin · 10.0.0.5



Пример: один шаблон, разные параметры

Зачем разбор помогает текстовой модели

Часть 3 · Блок 3 · Логи

Без парсинга три строки ниже – три разных «слова» для модели: они не совпадают посимвольно. После парсинга у них один шаблон – модель видит, что это один тип события, а различия вынесены в параметры.

```
Failed login for user admin from 10.0.0.5  
Failed login for user root from 192.168.1.7  
Failed login for user guest from 172.16.0.9
```

↓ log parsing

Один шаблон: Failed login for user <*> from <*>

Модель видит 3 события одного типа (а не 3 уникальных строки) + поле «параметры». Шум падает, сигнал концентрируется.



Одинаковый шаблон может значить разное

Часть 3 · Блок 3 · Логи

⚠ ЧАСТАЯ ОШИБКА

«Сгруппирую всё по шаблону – одинаковый шаблон значит одинаковое событие»

Не всегда. Один и тот же шаблон с разными параметрами может означать как фоновую активность, так и атаку. Параметры – не «мусор для выбрасывания», а часть смысла. Слепое объединение по шаблону скрывает аномалии.

Один шаблон – два разных смысла

Фоновая активность

Failed login for user <*> from <*>

user = опечатка сотрудника, from = офисный IP. Норма: 1-2 раза в день.

Признак атаки

Failed login for user <*> from <*>

user = admin, root, sa подряд, from = внешний IP. 400 раз за минуту – перебор паролей.



Мини-резюме блока 3

1

Лог = шаблон + параметры

У лог-сообщений двойная природа: постоянный шаблон-скелет и переменные значения. В естественном тексте такой структуры нет.

2

Парсинг снижает шум

Отделение шаблона от параметров превращает тысячи уникальных строк в немного типов событий – сигнал концентрируется.

3

Параметры – часть смысла

Один шаблон с разными параметрами может быть и нормой, и атакой. Выбрасывать параметры нельзя.

Дальше – блок 4: плотные представления. Embeddings и BERT-подобные модели на уровне применения.



БЛОК 4

Плотные представления

TF-IDF даёт разреженный вектор длиной во весь словарь. Embeddings идут другим путём: каждый текст – короткий плотный вектор, где близость означает близость смысла. Разбираем их на уровне применения – без углубления в архитектуру.

- Embeddings: что это и чем отличаются от TF-IDF
- Word, sentence, domain embeddings
- Transformer encoder и BERT – на уровне применения
- Дальше (часть 2 блока) – когда BERT не нужен и drift словаря



Embeddings: определение

34 / 53

Часть 3 · Блок 4 · Embeddings

Плотный вектор, где близость = смысл

ОПРЕДЕЛЕНИЕ

Embedding (эмбеддинг) – представление текста плотным вектором небольшой фиксированной размерности (например, 384 или 768 чисел), обученным так, что близкие по смыслу тексты получают близкие векторы. Расстояние между векторами отражает смысловую близость, а не совпадение слов.

Плотный

Почти все координаты ненулевые, в отличие от разреженного TF-IDF.

Фиксированной длины

Размерность не зависит от размера словаря – обычно сотни чисел.

Смысловой

Близость векторов = близость смысла.
«Письмо» и «сообщение» окажутся рядом.



Разреженное-прозрачное или плотное-смысловое

Это два принципиально разных способа представить текст. TF-IDF считает конкретные токены – вектор длинный, разреженный, каждая координата названа словом. Embedding кодирует смысл в короткий плотный вектор, где отдельная координата уже ничего не «значит» по отдельности.

TF-IDF

- Разреженный вектор длиной во весь словарь.
- Каждая координата = конкретное слово → прозрачно.
- Не понимает синонимов: «письмо» ≠ «сообщение».
- Чувствителен к новым словам (drift словаря).

Embeddings

- Плотный короткий вектор фиксированной длины.
- Координаты не интерпретируемы по отдельности.
- Улавливает смысловую близость и синонимы.
- Лучше переносится между похожими доменами.



Word, sentence и domain embeddings

36 / 53

Три уровня – на уровне различий

Часть 3 · Блок 4 · Embeddings

Эмбединги различают по масштабу и происхождению. Для прикладной работы важны три случая.

Word embeddings

Вектор для каждого слова отдельно. Текст – это набор векторов слов, которые нужно как-то агрегировать.

Sentence embeddings

Один вектор на всё сообщение или предложение. Удобно для классификации и поиска похожих текстов целиком.

Domain embeddings

Модель, дообученная на текстах конкретного домена (логи ИБ, тикеты SOC). Лучше ловит специфическую лексику.



Что они дают – без архитектуры

Современные sentence embeddings обычно строит transformer encoder – нейросеть, которая читает весь текст сразу и выдаёт контекстный вектор. BERT-подобные модели – самый известный пример. Для нас важно, что они дают на входе и выходе, а не как устроены внутри.

Связь с курсом. Общая структура нейросети – из L13. Transformer – это нейросеть особого вида; LLM как её развитие будут в L16.

Разбираем (уровень применения)

- Что подать на вход: текст целиком
- Что получить на выходе: один вектор
- Как использовать: классификация, поиск похожих

НЕ разбираем в этой лекции

- Внутреннюю архитектуру transformer
- Attention mechanism математически
- Fine-tuning и обучение LLM (это L16)



Две разные задачи на одних эмбедингах

Часть 3 · Блок 4 · Embeddings

Эмбединги особенно сильны там, где TF-IDF слаб: в поиске похожих текстов по смыслу. Но важно не путать две задачи – они требуют разной постановки, разных метрик и по-разному используют векторы.

Классификация

Отнести текст к классу

- Вопрос: «это фишинг или нет?»
- Нужны размеченные классы и обучение.
- Метрики: precision, recall, F1.

Поиск похожего

Найти близкие по смыслу случаи

- Вопрос: «какие инциденты похожи на этот?»
- Разметка классов не обязательна.
- Метрики: ранжирование, recall@k.

Поиск похожих инцидентов по описанию – типичная ИБ-задача, где эмбединги выигрывают у TF-IDF.



Частая ошибка: BERT всегда лучше

Закрываем линию *baseline*-дисциплины

Часть 3 · Блок 4 · Embeddings

⚠ ЧАСТАЯ ОШИБКА

«BERT – это нейросеть и SOTA, значит, он автоматически лучше линейки на TF-IDF»

Современность модели не гарантирует выигрыша на конкретной задаче. На коротких технических строках (URL, команды) и небольших корпусах простая линейка часто не уступает BERT – а иногда и превосходит его.

ПОЧЕМУ ЭТО НЕ ВСЕГДА ТАК

Стоимость

BERT требует GPU, больше памяти и времени. Линейка обучается за секунды на CPU.

Потеря прозрачности

У линейки видны веса токенов (L09). У эмбедингов координаты не интерпретируемы.

Риск переобучения

На малых ИБ-корпусах тяжёлая модель легко переобучается под артефакты данных.



Когда что выбрать – карта решений

Нет «лучшего» представления вообще – есть выбор под задачу. Четыре оси помогают решить осознанно.

Ось сравнения	TF-IDF + линейка	Embeddings / BERT
Качество	Сильное на коротких/технических текстах	Сильнее на связном смысловом тексте
Интерпретируемость	Высокая: веса токенов видны	Низкая: координаты не читаются
Вычислительная сложность	Низкая: CPU, секунды	Высокая: GPU, память, время
Переносимость	Хрупкая к drift словаря	Лучше переносится между доменами

Правило: начинать с TF-IDF + линейки; переходить к эмбедингам, только если они дают измеримый выигрыш, оправдывающий цену.



Язык атак меняется быстрее, чем модель

ОПРЕДЕЛЕНИЕ

Drift словаря (vocabulary drift) – ситуация, когда набор встречающихся токенов на проде систематически расходится со словарём обучающей выборки: появляются новые термины, домены, шаблоны кампаний, а старые выходят из употребления.

Почему в ИБ это особенно остро

Новые кампании

Каждая фишинговая волна приносит новые домены, бренды-приманки и формулировки.

Новая инфраструктура

Меняются IP-диапазоны, имена C2-серверов, пути и утилиты атакующих.

Новые техники

Свежие способы обфускации создают токены, которых не было в train.



Ошибка: не проверили drift словаря

42 / 53

Метрики на train отличные – в проде падают

Часть 3 · Блок 4 · Embeddings

Drift словаря тих и коварен: на обучающей и тестовой выборке всё хорошо, а на новых данных качество молча проседает, потому что половина важных токенов модели незнакома.

⚠ ЧАСТАЯ ОШИБКА

«Train/test split случайный, метрики высокие – модель готова к проду»

Случайный split перемешивает старые и новые данные, и drift не виден. Реальный прод получает данные из будущего – с новым словарём. Высокий offline-скор обманчив.

Что проверять

Доля незнакомых токенов на новых данных

Time-based split вместо случайного (как в L12)

Регулярное переобучение словаря



1

Embeddings – плотные и смысловые

Короткий вектор фиксированной длины, где близость означает близость смысла, а не совпадение слов.

2

BERT – не панацея

Современная модель не гарантирует выигрыша. Сравнивай с baseline по качеству, стоимости, прозрачности (линия baseline).

3

Классификация ≠ поиск

Эмбединги сильны в поиске похожего по смыслу, но это другая задача с другими метриками.

4

Drift словаря – реальная угроза

Язык атак меняется. Случайный split его скрывает; нужен time-based split (L12) и контроль новых токенов.

Основное содержание пройдено. Дальше – часть 4: разбор реальных ИБ-кейсов на текстовых данных.



Char n-grams + TF-IDF против эмбедингов

ИБ-КЕЙС

Задача. Отличить вредоносные URL от легитимных. Корпус ~50 000 ссылок, метки malicious / benign. URL почти всегда содержат несловарные токены: случайные поддомены, обфусцированные пути, подмены символов.

Подход А · char 3-grams + TF-IDF + линейка

- Символьные n-граммы ловят подмены и обфускацию.
- Обучение – секунды на CPU, веса токенов видны.
- $F1 \approx 0.96$ – сильный baseline.

Подход В · sentence embeddings

- Плотный вектор всего URL от предобученной модели.
- Дороже: нужна GPU, прозрачность теряется.
- $F1 \approx 0.965$ – выигрыш в пределах шума.

Вывод: на URL дешёвый char-baseline почти не уступает эмбедингам – выбираем его за скорость и прозрачность.



Почему символы и спецсимволы критичны

ИБ-КЕЙС

Задача. Выявить подозрительные командные строки процессов в телеметрии EDR. Метки: подозрительная / обычная активность.

```
powershell -nop -w hidden -enc SQBFAFgAKABOAGUAdwAt...
```

Сигнал – в символах и флагах

- Флаги -nop -w hidden -enc – маркеры скрытного запуска.
- Base64-хвост и регистр символов – часть сигнала.
- Char n-grams ловят это; словарь слов – нет.

Что убьёт стандартная очистка

- Нижний регистр сотрёт разницу hidden / Hidden.
- Удаление дефисов склеит флаги в кашу.
- Выброс base64 удалит сам payload-сигнал.



Чем они отличаются от обычного NLP

Часть 4 · Связки с ИБ

Текстовые данные безопасности отличаются от новостей и отзывов, на которых обычно учат NLP. Эти отличия определяют все решения – от токенизации до выбора модели.

Сигнал – в технических токенах

Важны не только слова, но символы, пути, флаги, параметры. Очистка должна быть осторожной.

Словарь быстро меняется

Новые кампании, домены и техники постоянно приносят невиданные токены – drift словаря неизбежен.

Логи имеют смешанную структуру

Шаблон плюс переменные параметры. Парсинг снижает шум, но параметры – часть смысла.



Самый коварный источник ложного качества

Часть 4 · Связки с ИБ

⚠ ЧАСТАЯ ОШИБКА

«Модель даёт $F1 = 0.99$ на текстах инцидентов – отличный результат!»

Подозрительно высокий скор на ИБ-текстах – частый признак утечки (data leakage, как в L05). В тексте остаётся след результата расследования, и модель «читает ответ», а не учится распознавать атаку.

ПОЧЕМУ ЭТО НЕВЕРНО

Идентификаторы кампаний

ID кампании или инцидента однозначно выдаёт класс – модель запоминает его.

Артефакты разметки

Пометки аналитика («confirmed», «TP») попали в текст и стали меткой класса.

Результаты расследования

Поля, заполненные ПОСЛЕ вердикта, недоступны в момент реального предсказания.



Когда модель ошибается, важно не просто увидеть метрику, а понять, на какие токены она реагирует. Это тот же error analysis, что и в L09, перенесённый на текст: разбираем конкретные ошибочные примеры и смотрим, какие токены утянули предсказание не туда.

Инструментарий – из L09. Error analysis и анализ важных признаков переносятся на токены текста без изменений.

Топ важных токенов

Осмысленные признаки атаки в топе → модель адекватна.
Идентификаторы и артефакты → сигнал утечки.

Разбор ошибочных примеров

Ложные срабатывания и пропуски: какие токены сбили модель. Часто вскрывает шум разметки и неоднозначность.



7 ключевых утверждений

- 1 Представление текста определяет всё: какой сигнал модель увидит, насколько она интерпретируема и как переживёт смену домена.
- 2 TF-IDF + линейная модель – обязательный baseline для текстов, прежде чем браться за эмбединги.
- 3 Чрезмерная очистка опасна: в ИБ символы, пути и флаги часто и есть признак атаки.
- 4 Логи = шаблон + параметры; парсинг снижает шум, но параметры – часть смысла.
- 5 Классификация и поиск похожего – разные задачи с разными метриками.
- 6 Веса токенов интерпретируемы – это инструмент проверки модели и обнаружения утечки (L09).
- 7 BERT не берётся по умолчанию: drift словаря и цена сложной модели реальны.



Карта решений по выбору представления

Короткая практическая памятка: какое представление брать под задачу, объём данных и требования к интерпретируемости.

Берите TF-IDF + линейку

- Короткие технические строки: URL, домены, команды.
- Малый или средний корпус.
- Нужна интерпретируемость (веса токенов).
- Ограничены по вычислениям (CPU, скорость).
- Всегда – как первый baseline.

Рассмотрите embeddings

- Связный смысловой текст: тикеты, отчёты, описания.
- Большой корпус, есть ресурсы (GPU).
- Задача поиска похожего по смыслу.
- Важны синонимы и перефразировки.
- Только после baseline и с измеримым выигрышем.



Откуда пришли и что читать дальше

L15 опирается на классические модели и интерпретируемость, а её темы продолжаютсся в блоке про надёжность.

L06

Линейная модель – baseline на TF-IDF

[назад](#)**L09**

Интерпретация важных токенов и ошибок

[назад](#)**L12**

Временная сторона логов и time-based split

[назад](#)**L16**

LLM как развитие transformer-моделей

[вперёд](#)**L18**

Текстовый adversarial – атаки на вход модели

[вперёд](#)**L19**

Текстовый poisoning – порча обучающих данных

[вперёд](#)



Конкретные главы и материалы

Обязательно

- Manning, Schütze. Foundations of Statistical NLP – гл. 5 (collocations), гл. 6 (statistical inference), гл. 14 (clustering).
- Bishop. Pattern Recognition and Machine Learning – гл. 4 (linear models for classification): база для линейного baseline.

Дополнительно

- Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers (2019) – для понимания BERT на уровне применения.
- Обзор log parsing: He et al. Drain – онлайн-парсер шаблонов логов (для блока про логи).

Практика

- Документация scikit-learn: TfidfVectorizer + LogisticRegression – собрать baseline-конвейер.
- Публичные корпуса: датасеты фишинговых писем и URL-классификации (Kaggle).



Ключевые термины лекции

Токен	минимальная единица текста: слово, символ или n-грамма	Sentence embedding	один вектор на всё сообщение целиком
n-грамма	последовательность из n подряд идущих токенов	Transformer encoder	нейросеть, дающая контекстный вектор текста
Bag-of-words	вектор частот слов; порядок отбрасывается	BERT	BERT-подобная модель для построения эмбеддингов
TF-IDF	взвешивание: частое в документе × редкое в корпусе	Log parsing	разбор лога на шаблон и переменные параметры
Baseline	опорное простое решение (здесь – TF-IDF + линейка)	Шаблон (template)	постоянный «скелет» лог-сообщения
Embedding	плотный вектор фикс. длины, близость = смысл	Drift словаря	расхождение токенов прода со словарём train